

Base de données orientée Colonnes

Juba AGOUN, PhD

Wide-column et base de données colonne

- Historiquement il y a eu un glissement de vocabulaire, et il existe en réalité deux familles de bases complètement différentes qui se disputent le mot « colonne ».

Les deux familles à ne pas confondre

Wide-column

Cassandra • HBase • ScyllaDB • Bigtable

Columnar / OLAP

ClickHouse • BigQuery • Redshift

Le mot « colonne » est trompeur : wide-column décrit surtout le modèle logique.

Wide-column

Wide-column

Cassandra • HBase • ScyllaDB • Bigtable

- **Les "wide-column stores" (stores orientés colonnes larges / column-family)**
 - c'est une table de hachage distribuée, persistante et multidimensionnelle et triée, indexée par une clé de ligne.

Le modèle de données n'est ni vraiment orienté ligne, ni vraiment orienté colonne
- Les colonnes sont regroupées en **column families (appellation abandonnée pour Table)**, et à l'intérieur d'une family, les données restent stockées ligne par ligne.
- Partitionnement Horizontal

Columnar

Columnar / OLAP
ClickHouse • BigQuery • Redshift

- **Les vraies bases orientées colonnes (OLAP columnar)**
 - chaque colonne est physiquement stockée séparément sur disque
- Partitionnement vertical

Wide-column ≠ base de données columnar

Concept	Wide-column	Columnar
Organisation	Partitions de lignes	Valeurs d'une même colonne regroupées
Objectif	OLTP distribué, écritures massives	OLAP, scans et agrégations
Accès typique	Par clé / partition	Par colonnes et filtres
Exemple	Cassandra	ClickHouse

Wide-column ≠ base de données columnar

Évolution par concept scientifique

2006 — papier Google *Bigtable* : modèle column family, carte multidimensionnelle (row, column, timestamp) → value. Partitionnement **horizontal** par plages de row keys, avec regroupement **vertical** des colonnes en familles pour limiter l'I/O.

2008 — HBase : implémentation directe du papier Bigtable au-dessus de HDFS.

2008 — Cassandra (Facebook) : fusionne le modèle de données de Bigtable (column family) avec l'architecture distribuée de Dynamo (Amazon) — hachage cohérent, quorum réglable. C'est là que la confusion "Cassandra = colonne" démarre.

~2013 (CQL3, Cassandra 1.2) — Cassandra abandonne le vocabulaire "column family" en surface au profit de "table" façon SQL

2005 — C-Store (MIT, papier académique fondateur du stockage colonne pur) → donne naissance à Vertica. Vulgarisation par Parquet/ORC pour le stockage fichier.

2016 — ClickHouse : vectorisation, compression par colonne. Pas de référence à Bigtable/Dynamo.

Pourquoi le concept de colonne quand même?!

Selon Basé sur le tutoriel VLDB09 (Abadi, Boncz & Harizopoulos)

- L'I/O en OLAP est optimal
- Compression Élevée du fait que les données homogènes du même type sont dans un bloc = taux de compression exceptionnel.
- Exécution Vectorisée : Les opérateurs traitent des paquets (vecteurs) de valeurs d'un coup (utilisation du cache CPU).
- Matérialisation Tardive - Les données restent compressées/séparées le plus longtemps possible dans le pipeline SQL.

Early vs Late Materialization

VLDB09 (Abadi, Boncz & Harizopoulos)

Pour traiter `SELECT A, B WHERE C > 100` :

Early Materialization

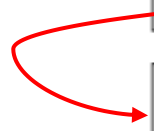
1. Assemble les tuples `(A, B, C)` immédiatement.
2. Applique le filtre sur `C`.
3. **Inconvénient** : Perte du traitement vectorisé et surcoût de mémoire.

Late Materialization

1. Filtre `C` et produit un masque de positions (bit-vector).
2. Utilise le masque pour extraire uniquement les positions valides dans `A` et `B`.
3. **Avantage** : Traitement CPU optimal.

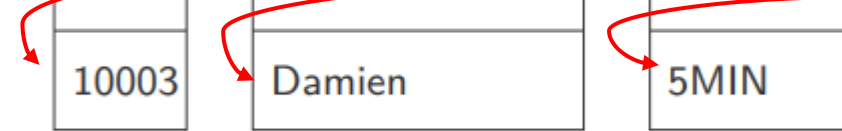
Ligne vs colonne

ID	Firstname	Class
16139	Alexis	3BE
10003	Damien	5MIN



Stockage de lignes

ID	Firstname	Class
16139	Alexis	3BE
10003	Damien	5MIN



Stockage de colonnes

- Stockage sur disque par tuples ou par colonnes
- Les requêtes ne concernent souvent pas toutes les colonnes
 - Récupération directe de colonnes depuis le disque plus efficace

Ligne vs colonne

- Choix du stockage sur disque pour l'efficacité des opérations
 - Stockage de lignes efficace pour les écritures
 - Stockage de colonnes efficace pour les lectures
- Lecture de quelques colonnes de beaucoup de lignes
 - Améliore les performances des requêtes de sélection

Stockage de lignes		Stockage de colonnes	
+	Facile d'ajouter un enregistrement		Seules les données voulues sont lues
-	Lecture de données inutiles		Écriture d'un tuple nécessite plusieurs accès



Wide-column

Introduction à Cassandra

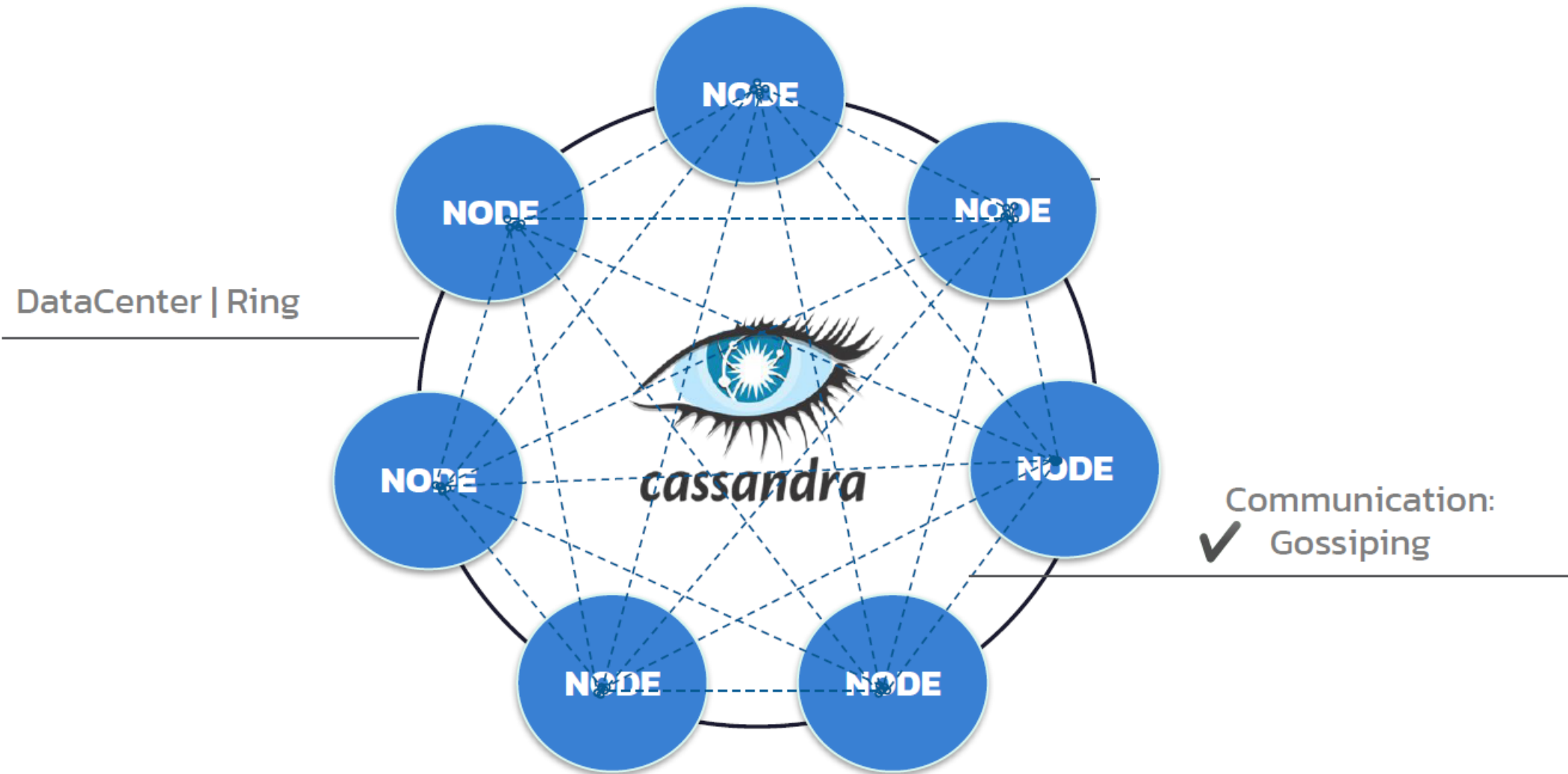
Bref historique

- 2007 : Conception originelle par les ingénieurs de Facebook pour des besoins internes
- 2009 : Cassandra est portée par l'Apache Incubator et sa communauté
- 2010 : promotion au rang des projet top-level
- juin 2011 (v0.8) : ajout du CQL
- depuis juin 2017 : version 3.11

État actuel

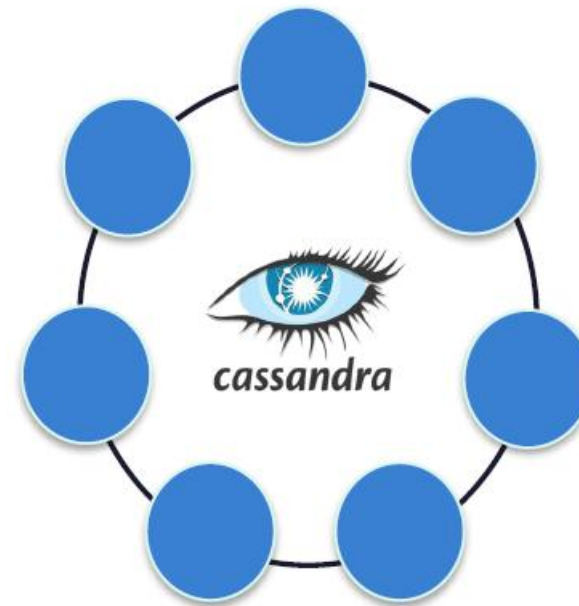
- Développeur : *fondation Apache.*
- Principal distributeur et contributeur : *société Datastax*
- Principaux utilisateurs : *Apple, CERN, Cisco, Netflix, Uber, ...*

Apache Cassandra - NoSQL Distributed Database

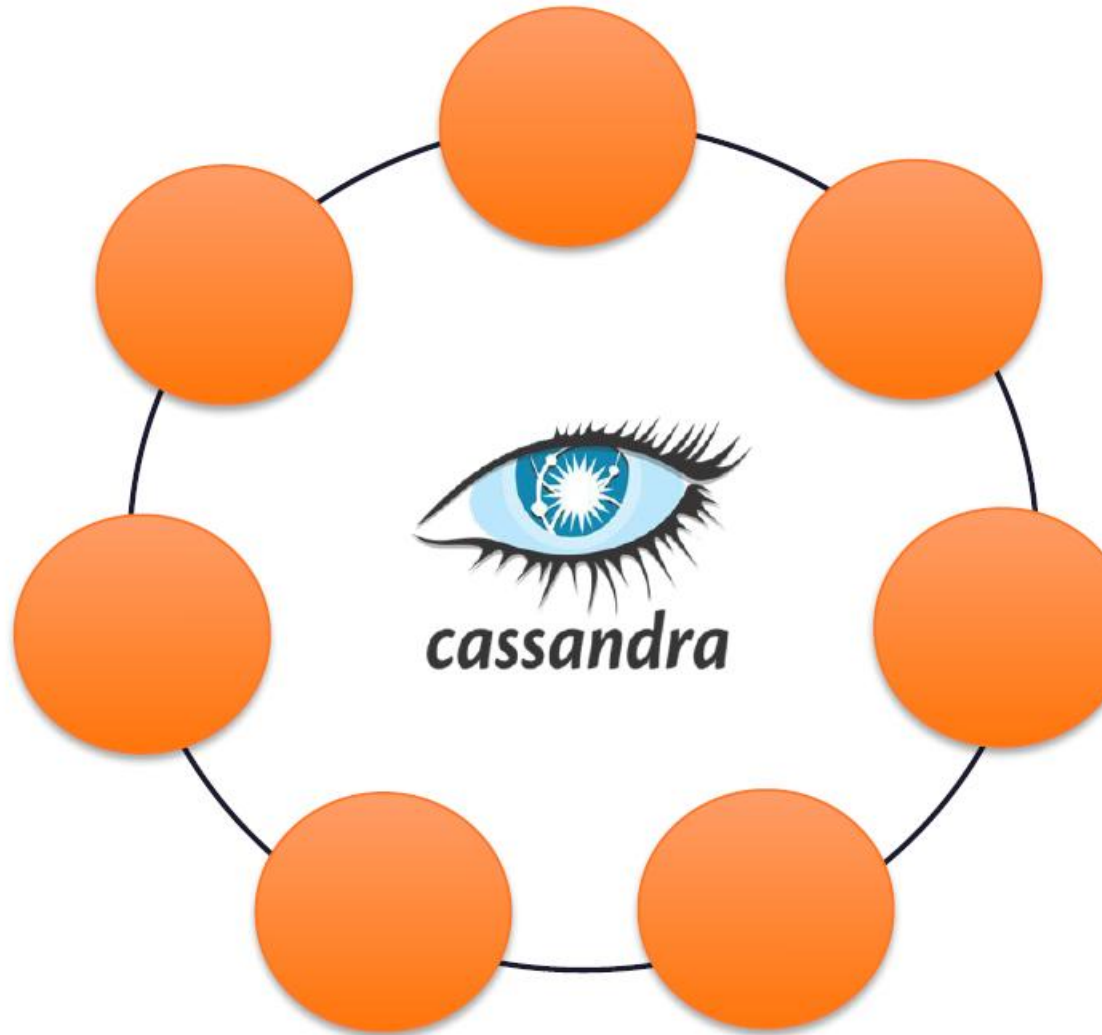


Apache Cassandra - NoSQL Distributed Database

- Disponibilité maximale
- Répartition géographique
- Performances en lecture/écriture



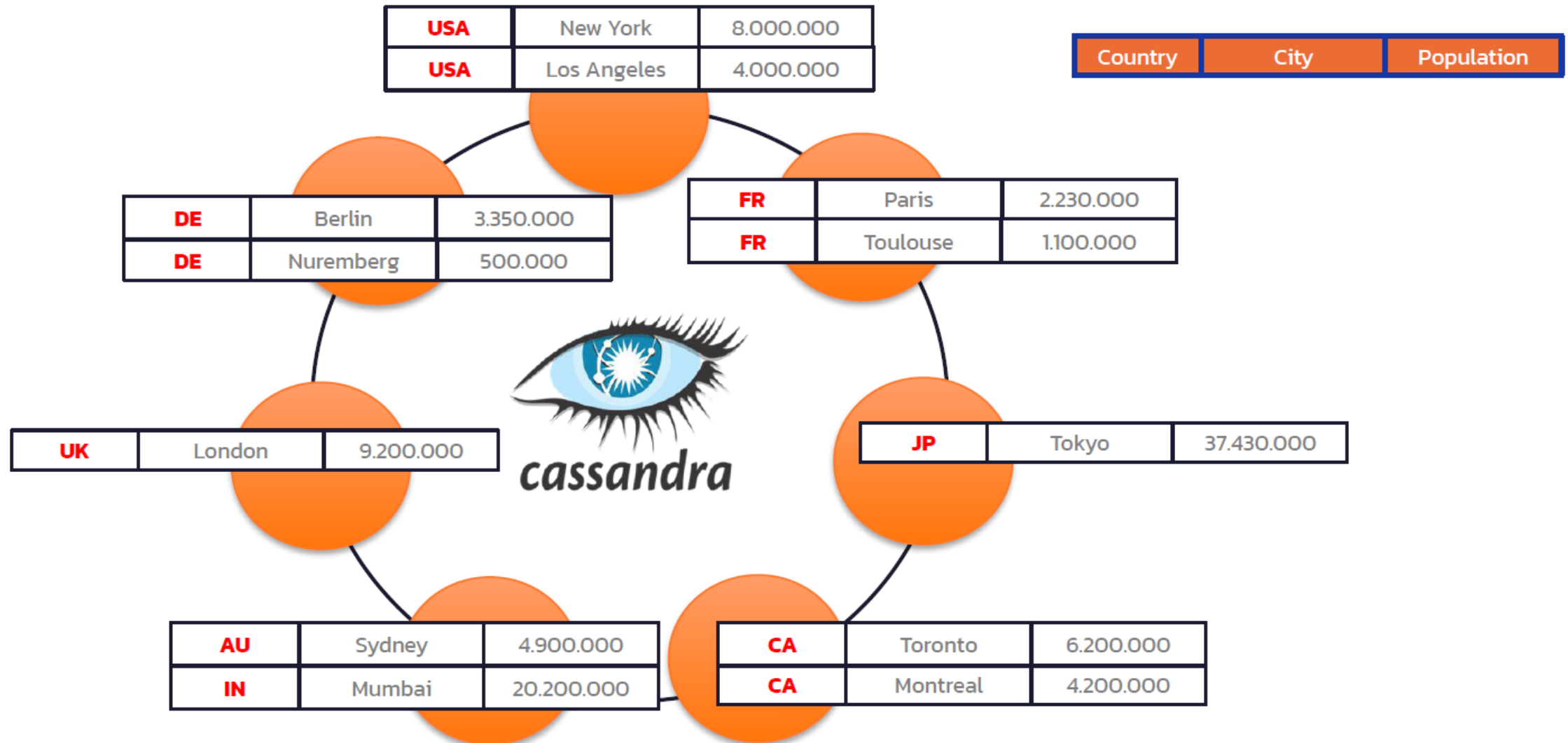
Apache Cassandra – Distribution des données



Country	City	Population
USA	New York	8.000.000
USA	Los Angeles	4.000.000
FR	Paris	2.230.000
DE	Berlin	3.350.000
UK	London	9.200.000
AU	Sydney	4.900.000
DE	Nuremberg	500.000
CA	Toronto	6.200.000
CA	Montreal	4.200.000
FR	Toulouse	1.100.000
JP	Tokyo	37.430.000
IN	Mumbai	20.200.000

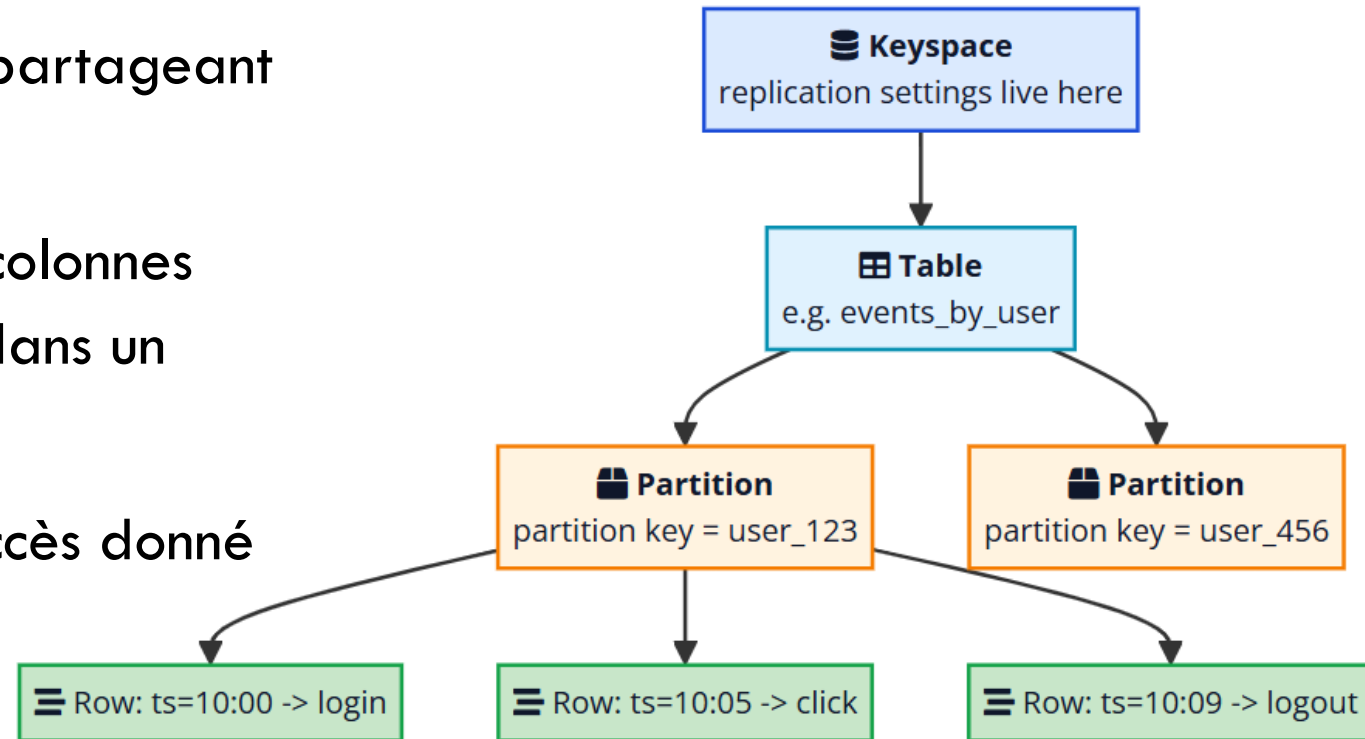

Partition Key

Apache Cassandra – Distribution des données



Le modèle de données : de la table à la partition

- Keyspace → tables → partitions → lignes.
- Une partition regroupe toutes les lignes partageant une même clé de partition.
- Une ligne peut contenir un ensemble de colonnes flexible : le schéma est plus souple que dans un modèle relationnel classique.
- La structure est conçue pour rendre un accès donné rapide et prévisible.



La structure de données

- Cellule : intersection d'une ligne et d'une colonne
- Une ligne : élément de données unique et structuré dans un tableau.
- Une partition : ensemble de lignes partageant le même jeton de partition, qui constitue l'unité de base d'accès dans Cassandra.
 - IMPORTANT : stockées ensemble, toutes ces lignes sont garanties d'être voisines.
- Une table : ensemble de colonnes et de lignes contenant des partitions.

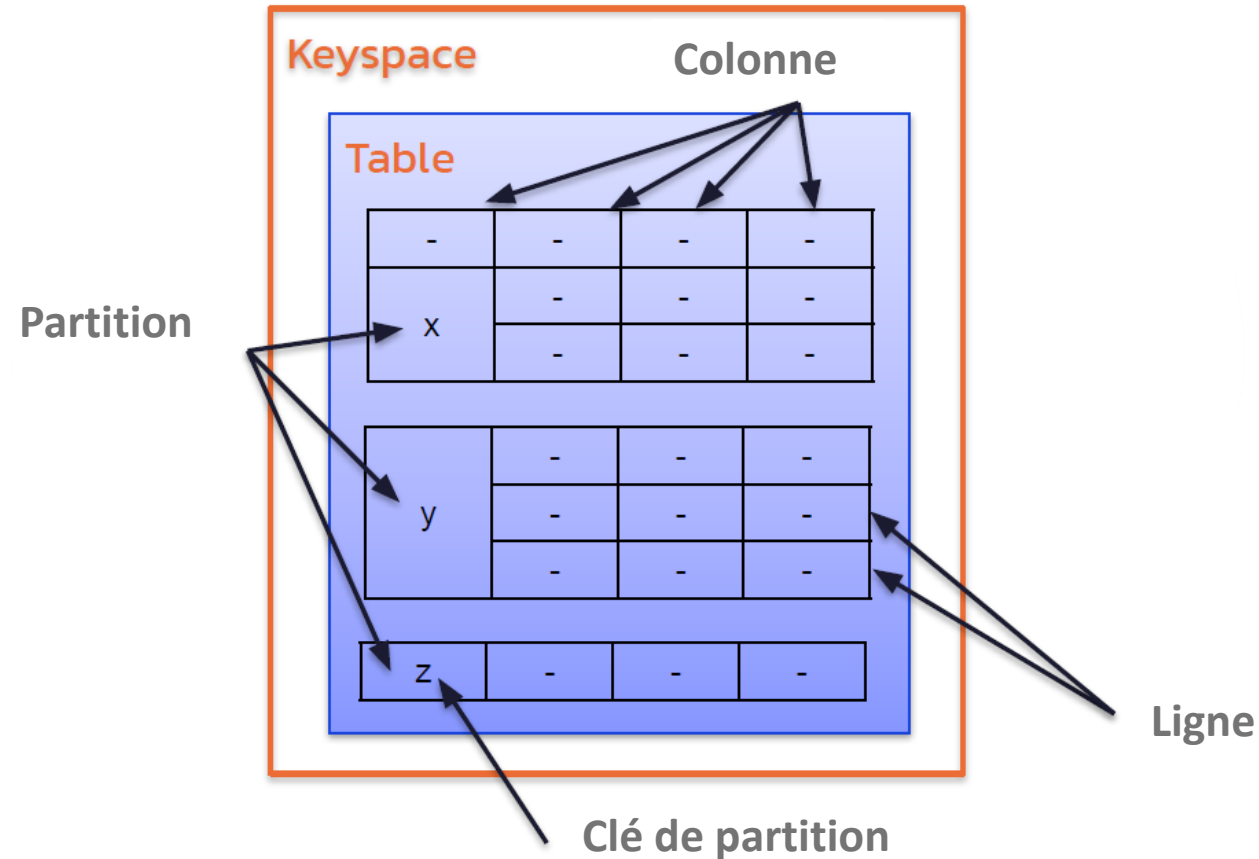
John

1	John	Doe	Wizardry
---	------	-----	----------

ID	First Name	Last Name	Department
1	John	Doe	Wizardry
399	Marisha	Chapez	Wizardry
415	Maximus	Flavius	Wizardry

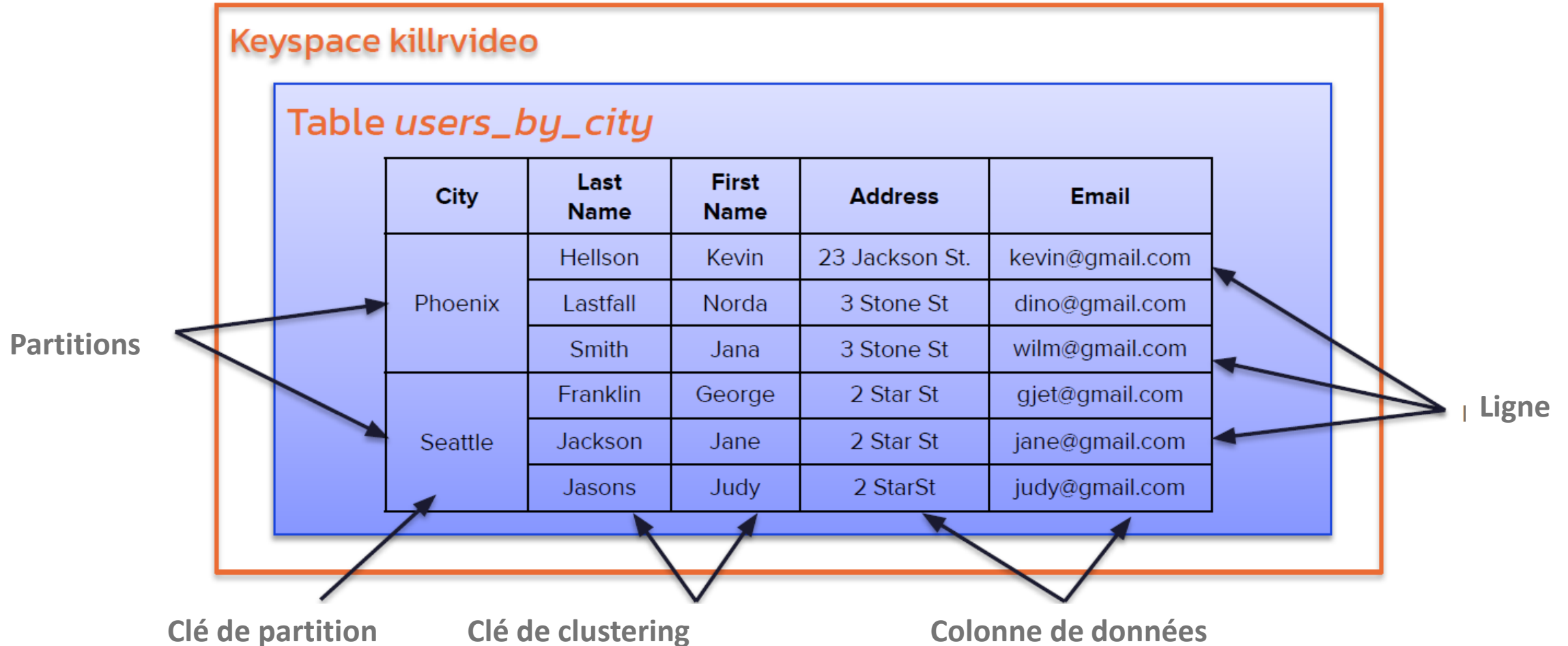
ID	First Name	Last Name	Department
1	John	Doe	Wizardry
2	Mary	Smith	Dark Magic
3	Patrick	McFadin	DevRel

Cassandra - Structure de données : Vue globale



- Modèle de données tabulaire, avec une particularité
- Les tables sont organisées en lignes et en colonnes
- Les groupes de lignes associées, appelés « partitions », sont stockés ensemble sur le ou les mêmes nœuds
- Chaque ligne contient une clé de partition
 - Une ou plusieurs colonnes qui sont hachées afin de déterminer le ou les nœuds sur lesquels ces données sont stockées

Exemple : Utilisateurs vu par ville



Cassandra : le langage CQL3

Qu'est ce que c'est ?

- un langage haut niveau de requêtage
- hérite de la syntaxe de SQL

Similaires avec SQL ?

- Data Definition Language = opération sur le schéma de données
⇒ CREATE, DROP, TRUNCATE, ALTER
- Data Manipulation Language = modification des données
⇒ INSERT, UPDATE, DELETE
- Data Retrieval Language = interrogation des données
⇒ SELECT

Interface client de Cassandra : le langage CQL3

Différences majeures avec SQL

- Ni jointure ni clés étrangères
 - ⇒ nécessité de dénormaliser le schéma relationnel
- Pas de GROUP BY
 - ⇒ doit être géré coté client (via traitement Map-Reduce)
- Clause WHERE limitée : pas de fonction de conversion ni de recherche contextuelle like %
- Clause ORDER BY limitée
- Pas de contrainte d'intégrité/d'unicité :
PRIMARY KEY précise juste la clé de partition de la table, mais aucune vérification n'est faite sur son unicité
- Les clauses INSERT et UPDATE sont équivalentes
- Valeur null ou vide non matérialisée.

Le modèle physique : CQL

Keyspace *Table*

```
CREATE TABLE killrvideo.users_by_city (  
  city text,  
  last_name text,  
  first_name text,  
  address text,  
  email text,  
  PRIMARY KEY ((city), last_name, first_name, email));
```

Colonnes

Clé primaire Clé de partition Clé de clustering

Clé primaire

- Identifiant d'une ligne. Se compose d'au moins une clé de partition et de zéro ou plusieurs colonnes de regroupement.

Important : IL FAUT S'ASSURER DE L'UNICITÉ.

Bon

```
PRIMARY KEY ((city), last_name, first_name, email);
```

```
PRIMARY KEY (user_id);
```

Mauvais

```
PRIMARY KEY ((city), last_name, first_name);
```

La méthode pour bien choisir est d'adopter une approche *query-first design*
(selon la requête de lecture)

Clé de clustering

- Used to ensure uniqueness and sorting order.

```
PRIMARY KEY ((city), last_name, first_name);
```

Non unique

```
PRIMARY KEY ((city), last_name, first_name, email);
```

okay

```
PRIMARY KEY ((video_id), comment_id);
```

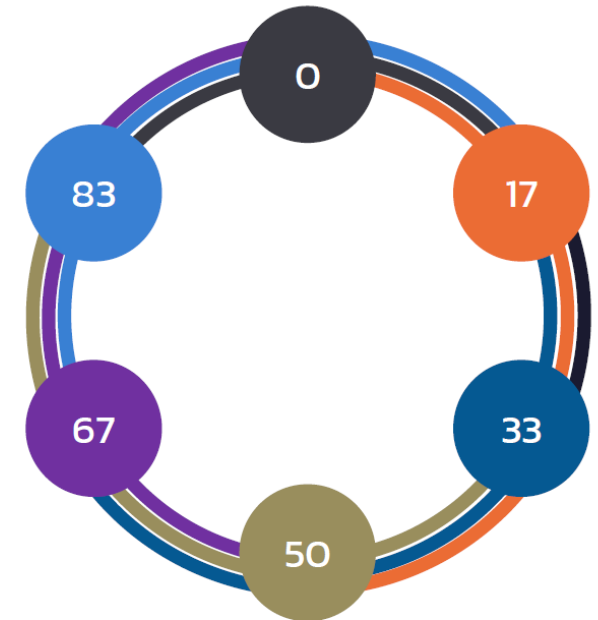
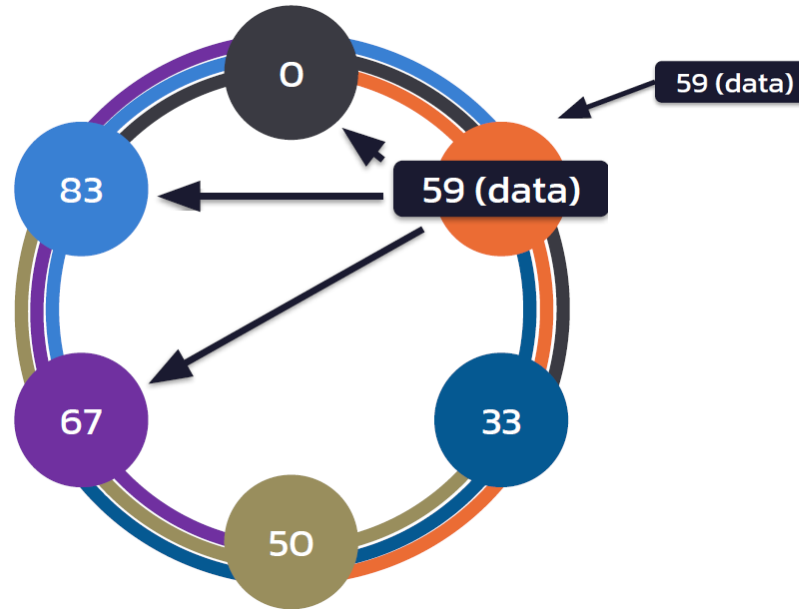
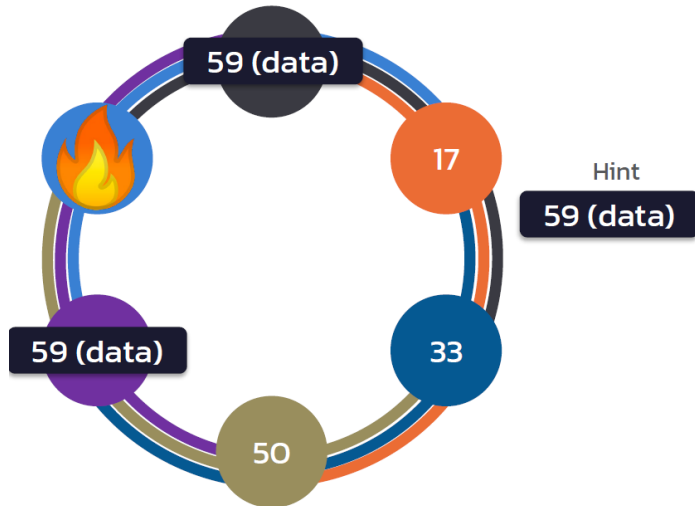
Relation d'ordre pas possible

```
PRIMARY KEY ((video_id), created_at, comment_id);
```

okay

Data is distributed, and also replicated

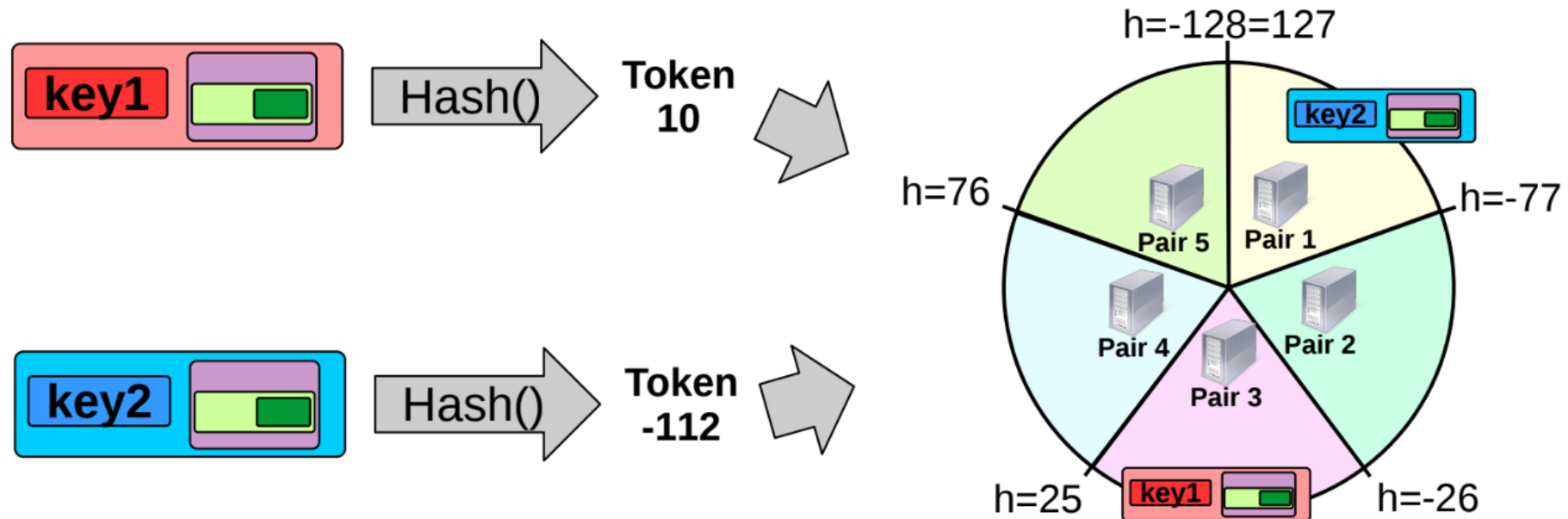
- Facteur de replication
 - Ex: $RF=3$
- En cas de panne d'un noeud



Partitionnement

Un principe similaire à une DHT - Distributed Hash Table -

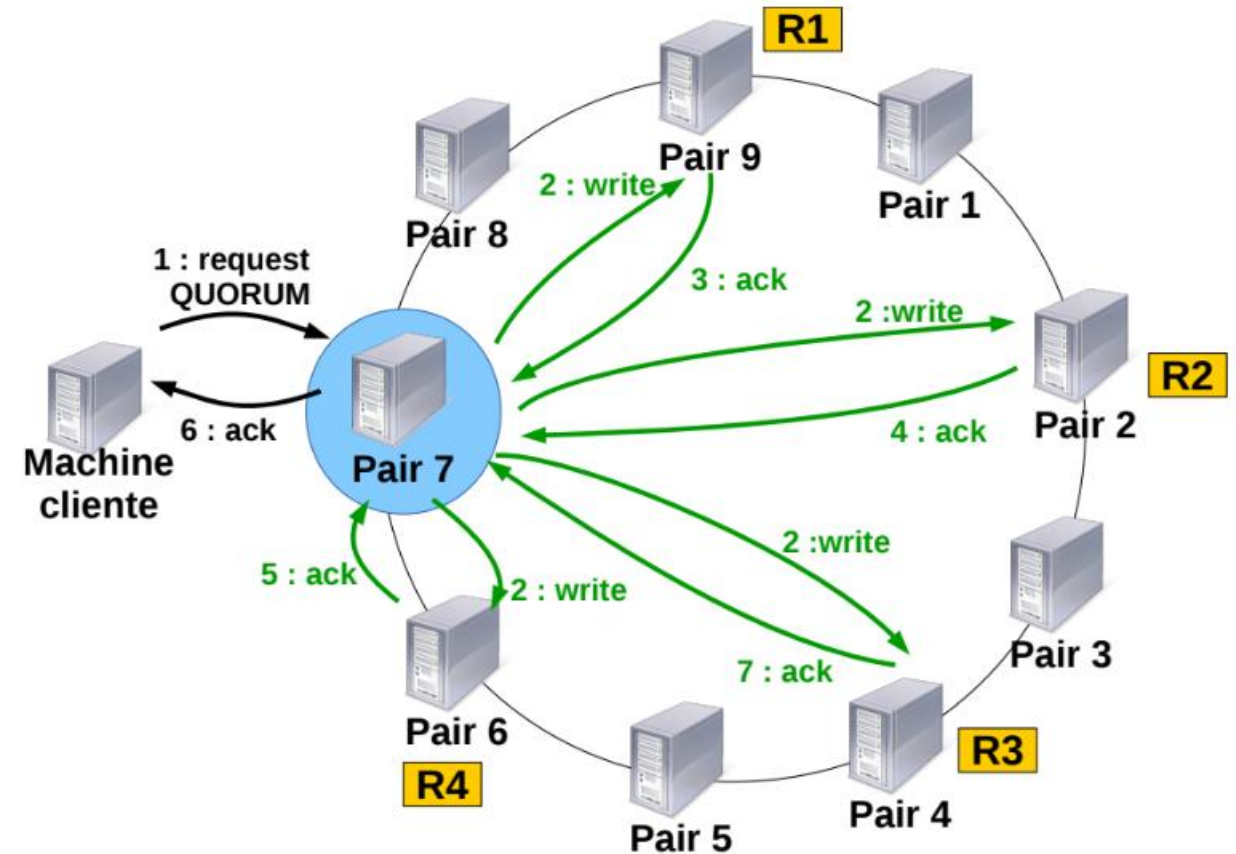
- Fonction de hachage calculant un token à partir de la clé de la ligne.
- Chaque nœud stocke un intervalle de valeur de token
- Dans un datacenter, les nœuds sont organisés logiquement en anneau sur l'espace de définition des valeurs de token



Requête d'écriture

Principes

- Le coordinateur envoie la requête à tous les réplicas
- Le niveau de cohérence = le nb de réplicas devant acquitter l'écriture
- La requête est terminée lorsque tous les **ack** requis ont été reçus



Requête de lecture

Principes

1. Le coordinateur envoie la requête aux nombres de réplicas voulus
⇒ on s'adresse aux réplicas les plus proches d'après le mécanisme de snitching.
2. Quand tous les résultats sont reçus, les comparer et vérifier leur cohérence
3. S'il existe différentes versions, renvoyer au client la version avec le timestamp le plus grand
4. En cas d'incohérence, le coordinateur envoie une requête `read_repair` aux réplicas pour mise à jour des obsolescences



ClickHouse

Columnar

ClickHouse ?

Pourquoi encore une nouvelle base de données ?



ClickHouse ?

Pourquoi encore une nouvelle base de données ?

- Quand les données deviennent très volumineuses, le problème n'est plus seulement de stocker les données, mais de pouvoir analyser rapidement des milliards de lignes.
- Bases relationnelles classiques :
 - excellentes pour les applications transactionnelles
 - nombreuses opérations Insert/update/delete (OLTP)
 - accès souvent orientés vers quelques lignes
- Mais pour l'analytique :
 - millions / milliards de lignes
 - agrégations massives
 - Que des :GROUP BY, SUM, AVG, COUNT

ClickHouse BD vraiment orientée colonnes

- Quand ClickHouse est une base de données analytique orientée colonnes.
- Elle est conçue pour scanner, filtrer et agréger de très grands volumes d'événements.
- OLTP : nombreuses petites lectures/écritures transactionnelles, avec cohérence immédiate.
- OLAP : questions analytiques portant sur de grandes quantités d'historique.
- Principe : conserver l'état transactionnel dans une base OLTP et envoyer les événements analytiques vers ClickHouse.



OLTP vs OLAP

- OLTP : commande, solde bancaire, profil utilisateur, panier...
- OLAP : nombre de requêtes échouées par heure, évolution des pages, agrégations historiques...
- ClickHouse est particulièrement adapté aux tableaux de bord.

Exemple de requête:

```
SELECT toStartOfDay(ts) AS day, count() AS pageviews
FROM events
WHERE ts >= now() - INTERVAL 30 DAY
GROUP BY day
ORDER BY day;
```

Pourquoi le stockage en colonnes ?

- Stockage par lignes : les valeurs d'un enregistrement sont regroupées.
- Stockage par colonnes : toutes les valeurs d'une même colonne sont regroupées.
- Une requête analytique lit souvent quelques colonnes sur beaucoup de lignes.
- ClickHouse peut donc ignorer les colonnes inutiles.

Même requête → beaucoup moins de données lues qu'avec un stockage par lignes.

- Colonnes lues / colonnes stockées:

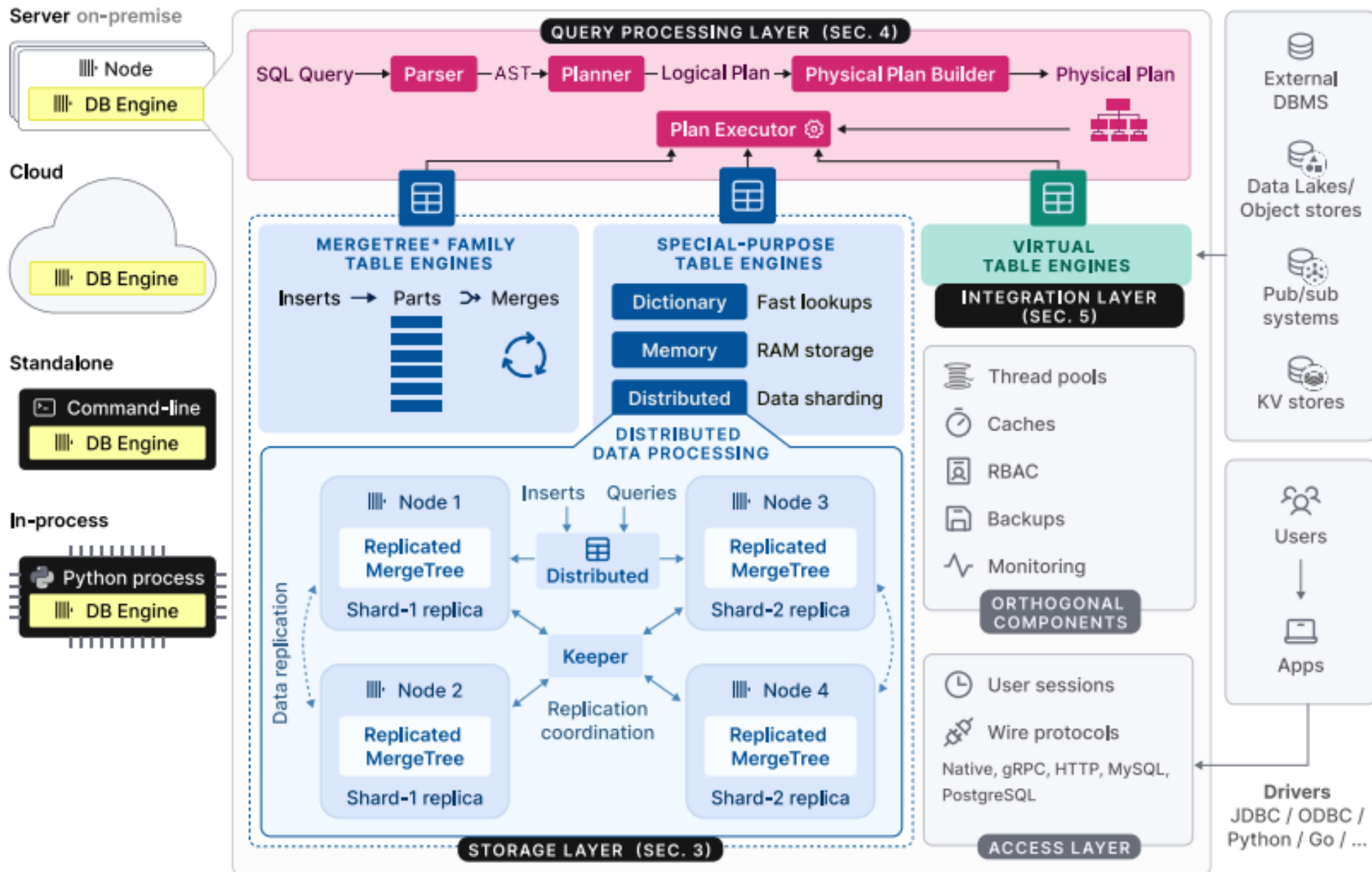
Table d'événements : par exemple 15 colonnes.

Requête analytique lisant 2–3 colonnes → le moteur évite de lire les 12–13 autres.

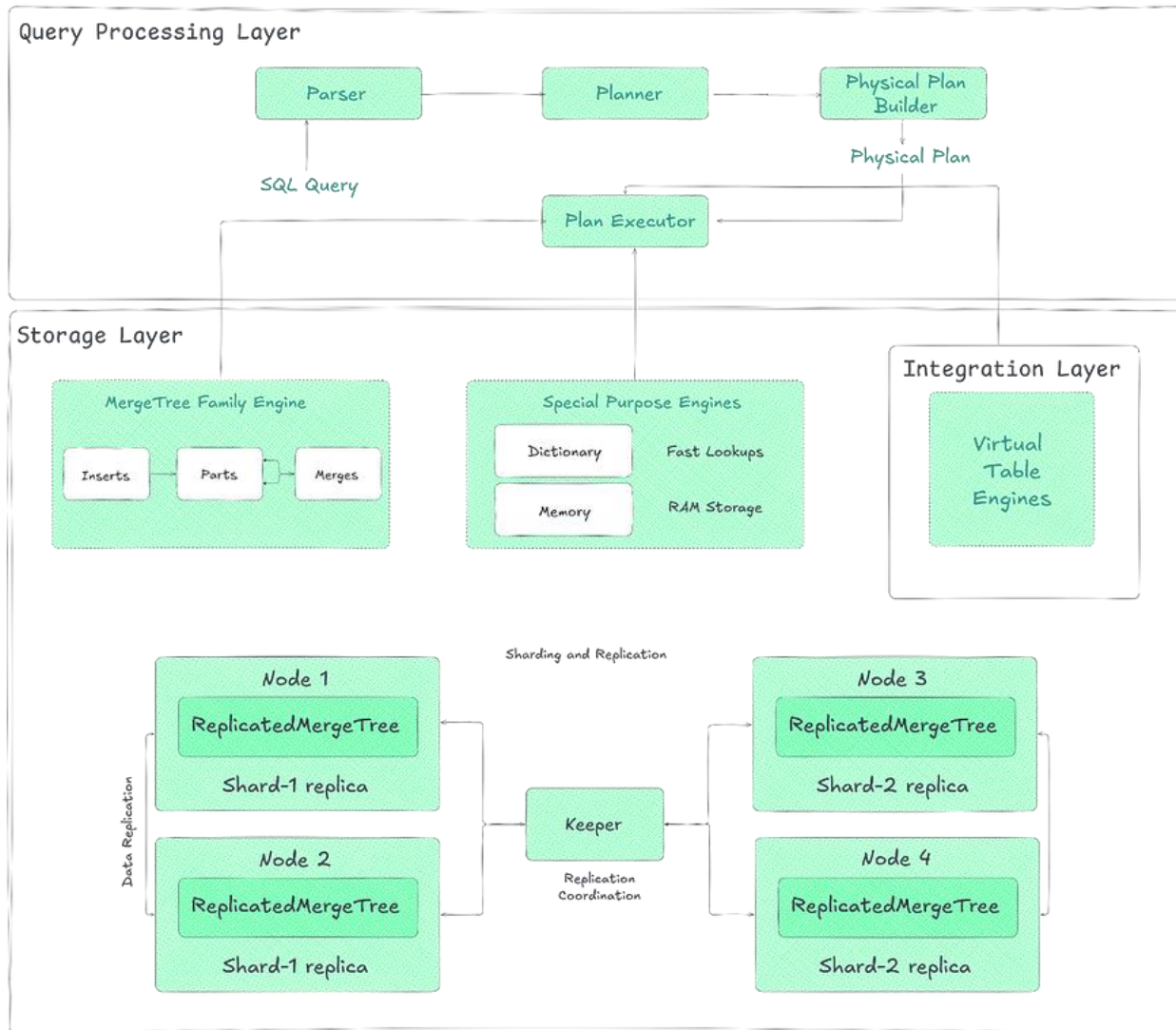
SELECT * annule une partie de cet avantage : chaque colonne demandée doit être lue.

La proportion de colonnes réellement lues est donc un facteur important de performance

Clickhouse Architecture



Clickhouse Architecture



Layer Storage : Stockage & Compression

- Stockage en colonnes : Lecture directe des colonnes ciblées sans scanner le reste.
- Compression optimisée
- Réduction de l'I/O : Volume de données divisé par deux, diminuant à la fois les coûts d'espace et les temps d'accès disques.

Query Execution Layer : Optimisation & Indexation

- Moteur d'exécution intelligent : Analyse de la requête et génération du plan le plus rapide
- Sparse Indexing (Index creux) : Indexation par blocs combinée à un tri par clé primaire.
- Skip Indexes : Ignore les blocs de données inutile lors de la lecture

Comment ClickHouse organise les données ?

Une table ClickHouse n'est pas une table classique!

- Cela dépende du moteur de table (type de table) détermine :
 - Comment et où les données sont stockées, où les écrire et d'où les lire.
 - Quelles requêtes sont prises en charge, et comment.
 - L'accès simultané aux données. L'utilisation des index, le cas échéant.
 - Si l'exécution multithread des requêtes est possible.
 - Les paramètres de réplication des données.

3 Catégories Principales de Table Engines

Famille MergeTree*

Moteur principal de stockage → données organisées en **parts triées**, fusionnées en arrière-plan (inspiré des **LSM-Trees**).

Dictionnaires & Caches

Moteurs spécialisés

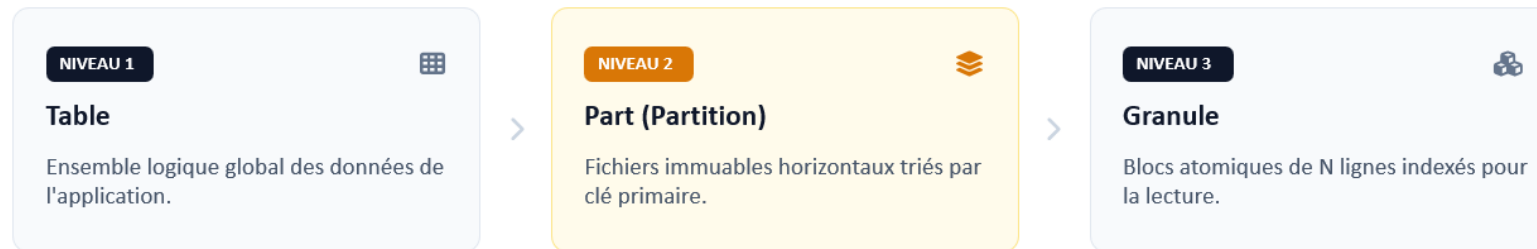
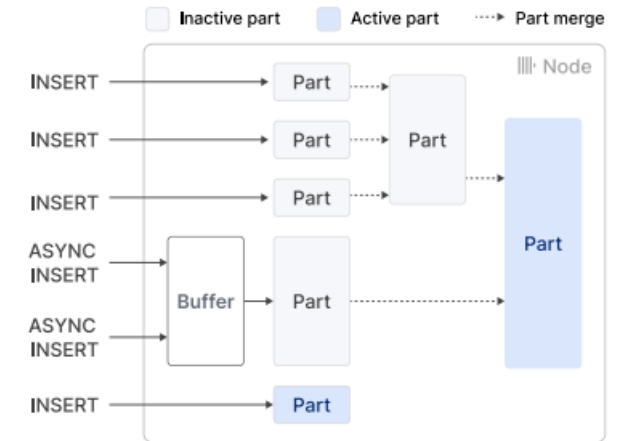
Optimisent l'exécution ou le stockage → **mémoire, dictionnaires (cache)**, ou **Distributed** pour le sharding.

Moteurs virtuels

permettent d'**échanger directement avec des systèmes externes** → PostgreSQL/MySQL, Kafka/RabbitMQ, Redis, Data Lakes ou stockage objet (S3, GCP...).

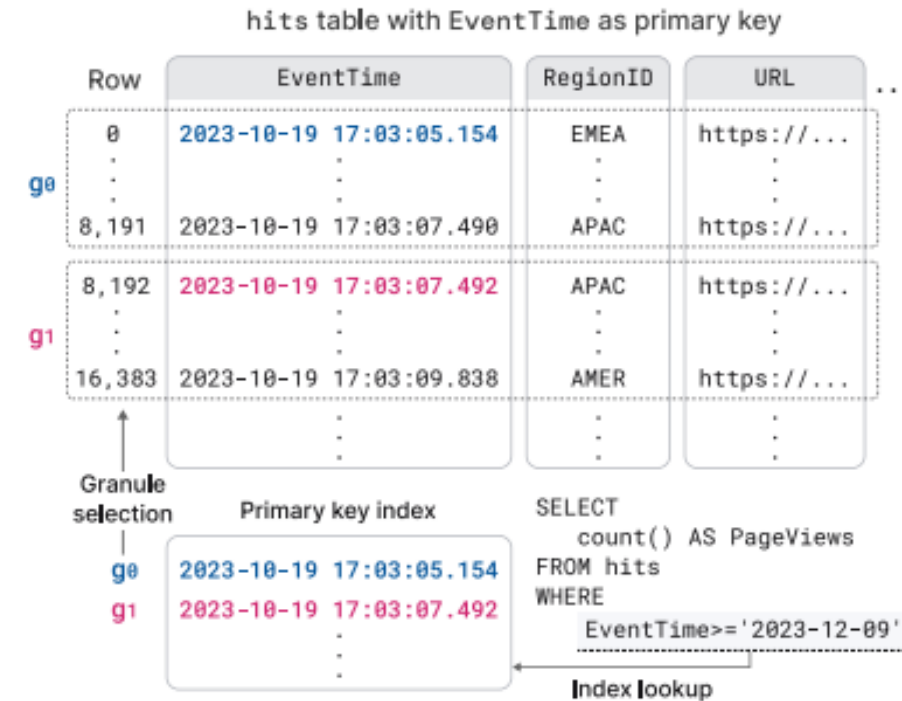
MergeTree : le modèle de stockage

- Chaque table ClickHouse déclare un moteur ; les tables analytiques utilisent très souvent la famille MergeTree.
- Chaque INSERT crée une nouvelle part, immuable.
- Une part contient les données organisées par colonnes.
- Des processus en arrière-plan fusionnent progressivement les petites parts.
- Les lectures peuvent continuer pendant ces fusions.



Évaluation des filtres avec un index sur la clé primaire

- Un index de clé primaire sur la colonne EventTime pour une table contenant des statistiques sur les impressions de pages.
- Les granules qui correspondent au prédicat de plage de la requête peuvent être trouvés par une recherche binaire dans l'index de clé primaire, au lieu d'un balayage séquentiel de la colonne EventTime.



Structure de données

```
CREATE TABLE events (  
    ts DateTime,  
    service LowCardinality(String),  
    user_id UInt64,  
    event_type LowCardinality(String),  
    duration_ms UInt32,  
    metadata String  
)  
ENGINE = MergeTree  
ORDER BY (service, ts);
```

➤ Choisir les types de données

- Choisir des types suffisamment précis, mais pas inutilement larges.
- UInt32 plutôt que UInt64 si la plage de valeurs le permet.
- LowCardinality(String) pour les colonnes contenant peu de valeurs répétées.
- Éviter Nullable lorsqu'une valeur par défaut claire suffit.

➤ ORDER BY : une décision physique

- ORDER BY définit l'ordre physique des lignes à l'intérieur de chaque part.
- Cet ordre alimente l'index primaire sparse.
- L'index permet de sauter des granules qui ne peuvent pas satisfaire un filtre.
- Une granule contient par défaut 8 192 lignes.

Conception : order by et partition

➤ **Order by comme clé primaire**

- Placer d'abord les dimensions fréquemment filtrées et relativement peu cardinales.
- Placer ensuite le temps lorsque les requêtes travaillent sur des intervalles temporels.
Exemple : ORDER BY (service, ts).
- Un user_id très cardinal en première position peut réduire l'efficacité du skipping.
- La bonne clé dépend des requêtes réellement exécutées : il faut la mesurer.

```
CREATE TABLE by_time (...)  
ENGINE = MergeTree ORDER BY ts;  
  
CREATE TABLE by_service (...)  
ENGINE = MergeTree ORDER BY (service, ts);
```

➤ **Partitions : pour le cycle de vie des données**

- Une partition regroupe des parts et constitue une unité de gestion.
- Elle sert surtout aux opérations de cycle de vie : rétention, déplacement ou suppression.
- Elle ne remplace pas ORDER BY pour accélérer les requêtes.
- Pour des données temporelles, une partition mensuelle est un choix courant.
- Éviter les clés de partition à très forte cardinalité : elles peuvent créer trop de partitions et trop de petites parts.

Conception : order by et partition

➤ **Order by comme clé primaire**

- Placer d'abord les dimensions fréquemment filtrées et relativement peu cardinales.
- Placer ensuite le temps lorsque les requêtes travaillent sur des intervalles temporels.
Exemple : ORDER BY (service, ts).
- Un user_id très cardinal en première position peut réduire l'efficacité du skipping.
- La bonne clé dépend des requêtes réellement exécutées : il faut la mesurer.

```
CREATE TABLE by_time (...)  
ENGINE = MergeTree ORDER BY ts;  
  
CREATE TABLE by_service (...)  
ENGINE = MergeTree ORDER BY (service, ts);
```

➤ **Partitions : pour le cycle de vie des données**

- Une partition regroupe des parts et constitue une unité de gestion.
- Elle sert surtout aux opérations de cycle de vie : rétention, déplacement ou suppression.
- Elle ne remplace pas ORDER BY pour accélérer les requêtes.
- Pour des données temporelles, une partition mensuelle est un choix courant.
- Éviter les clés de partition à très forte cardinalité : elles peuvent créer trop de partitions et trop de petites parts.

En résumé

Table

Partition

Order by ()

Granul

Granul

Granul

Partition

Order by ()

Granul

Partition = gros découpage

Part = morceau physique produit par les écritures/fusions

Granule = petite unité de lecture/saut

ORDER BY = organisation qui permet de savoir quels granules lire

Bravo! Vous avez réussi

Annexe (row oriented db)

rowid	id	first_name	last_name	ssn	salary	dob	title	joined
1001	1	John	Smith	111	101,000	1/1/1991	eng	1/1/2011
1002	2	Kary	White	222	102,000	2/2/1992	mgr	2/1/2012
1003	3	Norman	Freeman	333	103,000	3/3/1993	mkt	3/1/2013
1004	4	Nole	Smith	444	104,000	4/4/1994	adm	4/1/2014
1005	5	Dar	Sol	555	105,000	5/5/1995	adm	5/1/2015
1006	6	Yan	Thee	666	106,000	6/6/1996	mkt	6/1/2016
1007	7	Hasan	Ali	777	107,000	7/7/1997	acc	7/1/2017
1008	8	Ali	Bilal	888	108,000	8/8/1998	acc	8/1/2018

Annexe (row oriented storage)

1001, 1, John, Smith, 111, 101,000, 1/1/1991, eng, 1/1/2011 ||| •
1002, 2, Kary, White, 222, 102,000, 2/2/1992, mgr, 2/1/2012

1003, 3, Norman, Freeman, 333, 103,000, 3/3/1993, mkt, 3/1/2013 |||
1004, 4, Nole, Smith, 444, 104,000, 4/4/1994, adm, 4/1/2014

1005, 5, Dar, Sol, 555, 105,000, 5/5/1995, adm, 5/1/2015 |||
1006, 6, Yan, Thee, 666, 106,000, 6/6/1996, mkt, 6/1/2016

1007, 7, Hasan, Ali, 777, 107,000, 7/7/1997, acc, 7/1/2017 |||
1008, 8, Ali, Bilal, 888, 108,000, 8/8/1998, acc, 8/1/2018

Annexe (row oriented storage)

1001, 1, John, Smith, 111, 101,000, 1/1/1991, eng, 1/1/2011 ||| •
1002, 2, Kary, White, 222, 102,000, 2/2/1992, mgr, 2/1/2012

1003, 3, Norman, Freeman, 333, 103,000, 3/3/1993, mkt, 3/1/2013 |||
1004, 4, Nole, Smith, 444, 104,000, 4/4/1994, adm, 4/1/2014

1005, 5, Dar, Sol, 555, 105,000, 5/5/1995, adm, 5/1/2015 |||
1006, 6, Yan, Thee, 666, 106,000, 6/6/1996, mkt, 6/1/2016

1007, 7, Hasan, Ali, 777, 107,000, 7/7/1997, acc, 7/1/2017 |||
1008, 8, Ali, Bilal, 888, 108,000, 8/8/1998, acc, 8/1/2018

Annexe (row oriented storage)

Select first_name from emp where ssn=666

1001, 1, John, Smith, 111, 101,000, 1/1/1991, eng, 1/1/2011 |||
1002, 2, Kary, White, 222, 102,000, 2/2/1992, mgr, 2/1/2012



1003, 3, Norman, Freeman, 333, 103,000, 3/3/1993, mkt, 3/1/2013 |||
1004, 4, Nole, Smith, 444, 104,000, 4/4/1994, adm, 4/1/2014



1005, 5, Dar, Sol, 555, 105,000, 5/5/1995, adm, 5/1/2015 |||
1006, 6, Yan, Thee, 666, 106,000, 6/6/1996, mkt, 6/1/2016



Annexe (row oriented storage)

```
Select * from Emp where id = 1
```

1001, 1, John, Smith, 111, 101,000, 1/1/1991, eng, 1/1/2011 |||
1002, 2, Kary, White, 222, 102,000, 2/2/1992, mgr, 2/1/2012



Annexe (row oriented storage)

```
Select sum(salary) from emp
```

```
1001, 1, John, Smith, 111, 101,000, 1/1/1991, eng, 1/1/2011|||  
1002, 2, Kary, White, 222, 102,000, 2/2/1992, mgr, 2/1/2012
```

```
1003, 3, Norman, Freeman, 333, 103,000, 3/3/1993, mkt, 3/1/2013|||  
1004, 4, Nole, Smith, 444, 104,000, 4/4/1994, adm, 4/1/2014
```

```
1005, 5, Dar, Sol, 555, 105,000, 5/5/1995, adm, 5/1/2015|||  
1006, 6, Yan, Thee, 666, 106,000, 6/6/1996, mkt, 6/1/2016
```

```
1007, 7, Hasan, Ali, 777, 107,000, 7/7/1997, acc, 7/1/2017|||  
1008, 8, Ali, Bilal, 888, 108,000, 8/8/1998, acc, 8/1/2018
```

Annexe (row oriented storage)

Id	1:1001, 2:1002, 3:1003, 4:1004, 5:1005, 6:1006, 7:1007, 8:1008
F. Name	John:1001, Kary:1002, Norman:1003, Nole:1004, Dar:1005, Yan:1006, Hasan:1007, Ali:1008
S. Name	Smith:1001, White:1002, Freeman:1003, Sol:1004 Thee:1005, Ali:1006, Bilal:1007, Ali:1008
Ssn	111:1001, 222:1002, 333:1003, 444:1004, 555:1005, 666:1006, 777:1007, 888:1008
Salary	101000:1001, 102000:1002, 103000:1003, 104000:1004, 105000:1005, 106000:1006, 107000:1007, 108000:1008
DOB	1/1/1991:1001, 2/2/1992:1002, 3/3/1993:1003, 4/4/1994:1004, 5/5/1995:1005, 6/6/1996:1006, 7/7/1997:1007, 8/8/1998:1008
Position	eng:1001, mgr:1002, mkt:1003, adm:1004, adm:1005, mkt:1006, acc:1007, acc:1008
joined	1/1/2011:1001, 2/1/2012:1002, 3/1/2013:1003, 4/1/2014:1004, 5/1/2015:1005, 6/1/2016:1006, 7/1/2017:1007, 8/1/2018:1008

Annexe (row oriented storage)

Select first_name from emp where ssn = 666

Id

1:1001, 2:1002, 3:1003, 4:1004, 5:1005, 6:1006, 7:1007, 8:1008

F. Name

John:1001, Kary:1002, Norman:1003, Nole:1004, Dar:1005, Yan:1006, Hasan:1007, Ali:1008

S. Name

Smith:1001, White:1002, Freeman:1003, Sol:1004, Thee:1005, Ali:1006, Bilal:1007, Ali:1008

Ssn



11:1001, 222:1002, 333:1003, 444:1004, 555:1005, 666:1006, 777:1007, 888:1008



Salary

101000:1001, 102000:1002, 103000:1003, 104000:1004, 105000:1005, 106000:1006, 107000:1007, 108000:1008

DOB

1/1/1991:1001, 2/2/1992:1002, 3/3/1993:1003, 4/4/1994:1004, 5/5/1995:1005, 6/6/1996:1006, 7/7/1997:1007, 8/8/1998:1008

Position

eng:1001, mgr:1002, mkt:1003, adm:1004, adm:1005, mkt:1006, acc:1007, acc:1008

joined

1/1/2011:1001, 2/1/2012:1002, 3/1/2013:1003, 4/1/2014:1004, 5/1/2015:1005, 6/1/2016:1006, 7/1/2017:1007, 8/1/2018:1008

Annexe (row oriented storage)

Select * from emp where id = 1

Id	 1:1001, 2:1002, 3:1003, 4:1004, 5:1005, 6:1006, 7:1007, 8:1008
F. Name	 John:1001, Kary:1002, Norman:1003, Nole:1004, Dar:1005, Yan:1006, Hasan:1007, Ali:1008
S. Name	 Smith:1001, White:1002, Freeman:1003, Sol:1004, Thee:1005, Ali:1006, Bilal:1007, Ali:1008
Ssn	 111:1001, 222:1002, 333:1003, 444:1004, 555:1005, 666:1006, 777:1007, 888:1008
Salary	 101000:1001, 102000:1002, 103000:1003, 104000:1004, 105000:1005, 106000:1006, 107000:1007, 108000:1008
DOB	 1/1/1991:1001, 2/2/1992:1002, 3/3/1993:1003, 4/4/1994:1004, 5/5/1995:1005, 6/6/1996:1006, 7/7/1997:1007, 8/8/1998:1008
Position	 eng:1001, mgr:1002, mkt:1003, adm:1004, adm:1005, mkt:1006, acc:1007, acc:1008
joined	 1/1/2011:1001, 2/1/2012:1002, 3/1/2013:1003, 4/1/2014:1004, 5/1/2015:1005, 6/1/2016:1006, 7/1/2017:1007, 8/1/2018:1008